LinkedList Assignment Explanation

Source: Arizono

1. Overview

This project implements a singly linked list data structure in Java. The abstract class `AbstractList` and the `LinkedList` class act as containers that can store any object inheriting from the `Entity` parent class or passed as an `Object`. The `Burung` (Bird) and `Mobil` (Car) classes are subclasses of `Entity`. With this approach, a single linked list implementation can be used for various data types through the concept of polymorphism.

Project Structure

File	Role
`Entity.java`	Base class for stored data, containing a flexible `nilai` (`Object`) attribute.
`LinkedList.java`	Contains three main components: the `Node` class, the `AbstractList` abstract class, and the list implementation in `LinkedList`.
`Main.java`	Contains the subclasses `Burung` and `Mobil`, as well as the `Main` class to run and test the linked list functionality.

How to Run

```
javac *.java
java Main
```

Data Structure Illustration

The figure below illustrates the state of the list (`rantai`) after three objects are added in the `Main` class. The `head` pointer points to the first node, and the `next` reference of the last node is `null`.

```
head
 |
 v
Node: Burung("Elang") --next--> Node: Mobil("Amfibi") --next--> Node: Burung(99) -
-next--> null
```

2. Entity.java

A class that serves as the base element type. It stores a general value attribute to be inherited or used by other data types.

Entity.java, lines 1-6: attributes and constructor

```
public class Entity {
    Object nilai;

    public Entity(Object nilai) {
        this.nilai = nilai;
    }
```

- `Object nilai`: A general attribute to store data. It uses the `Object` type to accept any data type (e.g., String, Integer).
- `Constructor Entity(Object nilai)`: Initializes the `nilai` attribute when an object is instantiated.

Entity.java, lines 8-12: toString

```
    @Override
    public String toString() {
        return String.valueOf(this.nilai);
    }
}
```

- `toString()` is overridden to return the String representation of `nilai`. This method is called automatically when the object is printed using `System.out.print()`.

3. LinkedList.java

This file defines the node structure, the abstract list interface, and its concrete implementation.

LinkedList.java, lines 1-9: Node class

```
class Node {
    Object data;
    Node next;

    public Node(Object data) {
        this.data = data;
        this.next = null;
    }
}
```

- `Object data`: The data stored in the node.
- `Node next`: The pointer or link to the next node in the list.
- `Constructor Node(Object data)`: Initializes the new data and sets the reference to the next node as `null`.

LinkedList.java, lines 11-18: AbstractList

```
abstract class AbstractList {
    Node head;

    public abstract void insert(Object data);
    public abstract void tambah(Object data);
    public abstract void delete(Object data);
    public abstract void cetak();
}
```

- `Node head`: The head reference of the list that stores the first node. Because it is located in an abstract superclass without access modifiers, this attribute is package-private.
- Declares the abstract methods `insert`, `tambah`, `delete`, and `cetak`, forcing its subclasses to provide concrete implementations for these methods.

LinkedList.java, lines 20-27: LinkedList declaration and insert method

```
public class LinkedList extends AbstractList {

    @Override
    public void insert(Object data) {
        Node nodeBaru = new Node(data);
        nodeBaru.next = head;
        head = nodeBaru;
    }
```

- `LinkedList` inherits from `AbstractList`.
- `insert(Object data)`: Inserts data at the beginning (front/head). `nodeBaru.next` is linked to the old `head`, then the `head` pointer is shifted to `nodeBaru`. The time complexity is $O(1)$.

LinkedList.java, lines 29-41: tambah method

```
    @Override
    public void tambah(Object data) {
        Node nodeBaru = new Node(data);
        if (head == null) {
            head = nodeBaru;
            return;
        }
        Node sementara = head;
        while (sementara.next != null) {
            sementara = sementara.next;
        }
        sementara.next = nodeBaru;
    }
```

- `tambah(Object data)`: Adds a node at the end of the list.

- If the list is empty (`head == null`), `nodeBaru` immediately becomes the `head`.
- Otherwise, a temporary pointer (`sementara`) is used to traverse to the end of the list (`sementara.next != null`), and then `nodeBaru` is linked at the tail. The time complexity is $O(n)$.

LinkedList.java, lines 43-62: delete method

```
    @Override
    public void delete(Object data) {
        if (head == null) {
            return;
        }

        if (head.data.equals(data)) {
            head = head.next;
            return;
        }

        Node sementara = head;
        while (sementara.next != null && !sementara.next.data.equals(data)) {
            sementara = sementara.next;
        }

        if (sementara.next != null) {
            sementara.next = sementara.next.next;
        }
    }
```

- `delete(Object data)`: Deletes the first node that matches the given data.
- If the list is empty, it returns and does nothing.
- If the element to be deleted is located at the `head`, the `head` is shifted to `head.next`.
- To search in the middle or end: the pointer traverses as long as `next` is not `null` and the data in the next node does not match the parameter.
- Once the node is found, the link is bypassed (`sementara.next = sementara.next.next`), skipping the deleted node so it is removed from the chain.

LinkedList.java, lines 64-73: cetak method

```
    @Override
    public void cetak() {
        Node sementara = head;
        while (sementara != null) {
            System.out.print(sementara.data + " -> ");
            sementara = sementara.next;
        }
        System.out.println("null");
    }
}
```

- Prints all elements in the list sequentially. It traverses the list using a `while` loop until it reaches `null`, automatically calling the `toString()` method every time an object is printed.

4. Main.java

This file contains the specific subclass implementations (`Burung` and `Mobil`) and the `main` method that executes the list operations.

Main.java, lines 1-19: Subclasses (Burung and Mobil)

```
class Burung extends Entity {
    public Burung(Object nilai) {
        super(nilai);
    }

    public void terbang() {
        System.out.println("Burung [" + this.nilai + "] terbang.");
    }
}

class Mobil extends Entity {
    public Mobil(Object nilai) {
        super(nilai);
    }

    public void berenang() {
        System.out.println("Mobil [" + this.nilai + "] berenang.");
    }
}
```

- Both of these classes are subclasses of `Entity` that call the parent constructor using `super(nilai)`.
- Each has its own unique method: `terbang()` (fly) for `Burung` and `berenang()` (swim) for `Mobil`. Although they are not explicitly called in this `main` method, they define the potential behavior of these objects.

Main.java, lines 21-31: main Method and List Initialization

```
public class Main {
    public static void main(String[] args) {
        LinkedList rantai = new LinkedList();

        rantai.tambah(new Burung("Elang"));
        rantai.tambah(new Mobil("Amfibi"));
        rantai.tambah(new Burung(99));

        rantai.cetak();
    }
}
```

- Creates a list object named `rantai`.
- Inserts three objects using the `tambah` method: a `Burung` (with a String value), a `Mobil`, and another `Burung` (with an Integer value). This proves the flexibility of using the generic `Object` type.
- `rantai.cetak()` is called to print the current state of the list to the terminal. The program finishes its execution right after printing the list.

5. Program Output

Because the downcasting and subclass-specific method calls have been removed, here is the simplified result that will be printed on the console when the compiled code is executed:

```
Elang -> Amfibi -> 99 -> null
```